

SUMMER COURSE

From Behavior Cloning to VLA

Week 2 · 模型、梯度、序列與多模態，看懂一個 VLA policy 的骨架

獵奇緯



目錄

01 課程地圖

03 Part 1 · 最小 Supervised Learning
骨架

05 Part 3 · 視覺表示

07 課後延伸

02 Part 0 · Behavior Cloning 是什麼
問題

04 Part 2 · 序列化 BC

06 Part 4 · VLA-shaped BC

SECTION 01

課程地圖



01

I 你要能看懂什麼

一個 *VLA policy* 的骨架：

資料流、*loss*、*token*、*attention*、*mask*、*encoder*、*decoder* 各自扮演什麼角色。

Vision

影像 → tokens · vision
encoder

Language

指令 → tokens ·
language encoder

Action

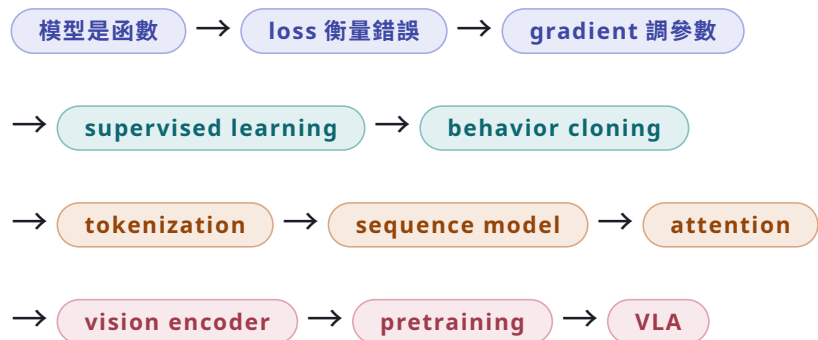
歷史 → 下一步 · action
decoder

Info

V + L + A 三條輸入，最後融合成一個 policy。這就是 VLA 的全名：Vision-Language-Action。

依賴圖

概念依賴鏈



✓ 今天的分段

- 0 BC 是什麼問題
- 1 supervised learning 骨架
- 2 序列化 BC
- 3 視覺表示
- 4 VLA-shaped BC
- 5 真實系統定位

SECTION 02

Part 0 · Behavior Cloning 是什麼問題



02

一批示範資料

$$D = \{(s_i, a_i)\}_{i=1}^N$$

 **s_i · state**

當時 **世界長什麼樣**：
相機影像、關節角度、紅綠燈的顏色.....

 **a_i · action**

demonstrator **在那個 state 下做的動作**：
方向盤角度、滑鼠座標、手臂位移.....

❶ Info

Behavior cloning 要學一個 policy：看到 **相似 state** 時，做出跟 demonstrator **相似的 action**。

兩種 policy 寫法



action 是連續值

滑鼠座標、手臂末端位置

$$f_{\theta}(s) = \hat{a}$$

直接輸出一個 **數值**



action 是離散選項

「下一個 action token 是哪一類」

$$\pi_{\theta}(a \mid s)$$

輸出「給定 state 之後，**每個 action 的機率**」

✓ Tip

這兩種形式今天都會用到：Part 1 的迴歸 vs 分類、Part 5 的 ACT vs RT-2，分野就從這裡開始。

條件機率

state	P(踩煞車 s)	P(踩油門 s)
 紅燈	0.95	0.05
 綠燈	0.10	0.90

讀表規則

- 每個 state 一列
- 同一列對所有 action **加總為 1**
- π_θ 跟這張表是 **同一個數學物件**

⚠ 注意

真實機器人的 state 空間太大、甚至連續，表格枚舉不了，所以改用一個帶參數 θ 的 **函數**：輸入 state、輸出那一列機率。

I θ : 模型裡所有可訓練的旋鈕



小模型

像 **收音機**

只有幾顆旋鈕



大模型

像 **錄音室控制台**

有幾十億顆旋鈕

機器再大，「根據錯誤把旋鈕轉到比較好的位置」這件事沒有變。

BC 的訓練目標

老師做了什麼，模型就 **提高那個答案的分數**，或讓輸出 **更接近那個 action**。

連續 action · MSE

$$L(\theta) = \frac{1}{N} \sum_i \|f_{\theta}(s_i) - a_i\|^2$$

預測跟示範的 **距離平方**

離散 action · NLL

$$L(\theta) = -\frac{1}{N} \sum_i \log \pi_{\theta}(a_i | s_i)$$

模型給 `a_i` 的機率越高，loss 越小

✓ Tip

第二個式子現在先看精神就好，等 Part 1 講完 softmax 和 cross-entropy，它會變得完全自然。

| BC 不是 RL



Reinforcement Learning

agent 透過 **reward**、**trial and error**、**policy improvement** 學策略

沒有人告訴你正確答案，只有分數



Behavior Cloning

就是 **supervised learning**：
每筆資料直接告訴模型「老師做了什麼」

答案已經在資料裡

① Info

很多 VLA 的第一階段訓練就是大規模 BC，之後才可能接 RL fine-tuning、preference optimization、真機修正或安全約束。

SECTION 03

Part 1 · 最小 Supervised Learning 骨架



Lab 1：企鵝的迴歸與分類



basic_regression_classification

資料：**Palmer Penguins**

- 輸入：嘴喙長度、嘴喙深度、鰭長等量測值
- 輸出：species（分類）或體重（迴歸）

這份 lab 只考慮一件事

feature vector 進



一個答案出

Info

還沒有序列、沒有影像、沒有語言，只是把「單步輸入到輸出」的骨架搭穩。

■ 線性模型：一次矩陣乘法

$$\hat{y} = Wx + b$$

x

feature vector
輸入的量測值

W

矩陣
模型的主要參數

b

bias
平移項

① Info

嚴格說這是 **affine map**：線性變換加一個平移.....習慣上大家仍叫它線性模型。

手算一次 forward pass

輸入與參數

$$x = [\text{嘴喙長}, \text{嘴喙深}] = [45, 15]$$

$$W = \begin{pmatrix} 0.20 & -0.10 \\ -0.05 & 0.30 \\ 0.10 & 0.05 \end{pmatrix}, \quad b = \begin{pmatrix} -7.0 \\ -2.0 \\ -5.0 \end{pmatrix}$$

$$z = Wx + b$$

$$z_1 = 0.20 \cdot 45 - 0.10 \cdot 15 - 7.0 = \mathbf{0.5}$$

$$z_2 = -0.05 \cdot 45 + 0.30 \cdot 15 - 2.0 = 0.25$$

$$z_3 = 0.10 \cdot 45 + 0.05 \cdot 15 - 5.0 = 0.25$$

第一類 logit 最大 → 預測 **第一類**

矩陣乘法



加 bias



選最大值

= 一次 forward pass

W 的每一列是一個模板

$$z_k = w_k \cdot x + b_k, \quad w_k \cdot x = \|w_k\| \|x\| \cos \theta$$

row-as-template

第 k 列 w_k = 第 k 類的 **模板向量**

norm 相當的前提下，內積大 = 輸入方向跟模板方向 **比較像**

這個觀念今天出現三次

- 1 線性分類器：class template · 輸入
- 2 CNN：kernel · local patch
- 3 attention：query · key ($QK^\top$)

決策邊界：一條線、一個超平面

兩個模板分數 **打平** 的地方： $(w_1 - w_2) \cdot x + (b_1 - b_2) = 0$

幾何意義

2D 是一條 **線**，高維是 **hyperplane**

線性模型的假設：資料可以被平面切開

假設的兩面

✓ 假設符合資料：穩、簡單、可解釋

✗ 假設不符合：**它就會卡住**

XOR：線性模型的天花板

x_1	x_2	target
0	0	0
1	0	1
0	1	1
1	1	0



30 秒的挫折

在平面上畫這四個點，
試著用 **一條直線** 把 0 和 1 分開。

畫不出來。

對角的兩對點，任何直線都切不開，這就是 MLP 的動機。

MLP：疊層，但關鍵是非線性

MLP：線性層 + activation 疊起來

$$h_1 = \sigma(W_1x + b_1)$$

$$h_2 = \sigma(W_2h_1 + b_2)$$

$$\hat{y} = W_3h_2 + b_3$$

沒有 σ 會發生什麼

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

疊再多層也 **塌回單一 affine map**，
表達能力跟一層完全相同

⚠ 注意

所以關鍵不是「深」，是 **非線性**。沒有 activation 的深度是假的深度。

ReLU 把空間摺起來

ReLU

$$\text{ReLU}(z) = \max(0, z)$$

$z < 0$ 輸出 0； $z > 0$ 原樣通過。

在 0 有一個 **轉折**，轉折讓模型能把空間切開、摺過去。

手算 XOR 摺疊： $h_1 = \text{ReLU}(x_1+x_2)$, $h_2 = \text{ReLU}(x_1+x_2-1)$

(x_1, x_2)	$h = (h_1, h_2)$	$g(h) = h_1 - 2h_2 - 0.5$
(0,0)	(0, 0)	-0.5
(1,0)	(1, 0)	+0.5
(0,1)	(1, 0)	+0.5
(1,1)	(2, 1)	-0.5

✓ Tip

原本在 x 空間不存在的分隔超平面，經過 **一次 ReLU 摺疊** 之後就存在了，正負剛好對應 XOR。

表達力夠了， θ 還是亂數

MLP 給了 f_θ 足夠的表達力，但 θ 還是亂數，它現在連企鵝都分不了。

① loss

先定義 **什麼叫錯**

② gradient

錯了 **往哪裡改**

③ optimizer

實際 **走一步**

④ validation

檢查是 **學會** 還是
背熟



輸入 x : 「我不是企鵝！」



$f_\theta(x)$: 「企鵝。信心 97%。」

Loss (迴歸) : MSE

$$L(\theta) = \frac{1}{N} \sum_i \|f_{\theta}(x_i) - y_i\|^2$$

平方的效果 ①

正負誤差 **不抵消**

+3 和 -3 不會互相騙過去

平方的效果 ②

大錯被 **懲罰得更重**

錯 2 倍 → 罰 4 倍

Loss (分類) : softmax + cross-entropy

softmax : logits $\rightarrow$ 機率

$$p_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}$$

所有 p_k 都是正數，總和是 1

cross-entropy · one-hot target

$$H(q, p) = - \sum_k q_k \log p_k \Rightarrow L = - \log p_y$$

target 在真實類別 y 上機率 1，整個和 **只剩一項**

✓ **Tip**

把正確類別機率 p_y 拉高 $\rightarrow$ loss 變小

✗ **危險**

很有自信地把 p_y 壓低 $\rightarrow$ loss 爆大

回頭看 BC 的 loss

$$L(\theta) = -\log \pi_{\theta}(a_i \mid s_i)$$

Part 0 回顧

這就是 cross-entropy：

示範者做了 a_i ，我們懲罰模型

沒有把 a_i 的機率拉高

對應關係

分類	BC
class y	demonstrator 的 a_i
輸入 x	state s_i
$-\log p_y$	$-\log \pi_{\theta}(a_i \mid s_i)$

Gradient descent：往下坡走

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} L(\theta^{(t)}), \quad t = 0, 1, 2, \dots$$

∇L

每顆參數的**偏導數**
排成一個向量

-

分量為正往左走
分量為負往右走

η · learning rate

一步跨多大
第一個
hyperparameter

Info

這是一個**遞迴定義的數列**：給定 $\theta^{(0)}$ ，反覆套用同一條公式就產生下一個 $\theta^{(t+1)}$ ，這就是「訓練」。

一顆參數的手算

設定

模型 $\hat{y} = wx$ ，資料 $(1, 2), (2, 4), (3, 6)$

正確關係是 $y = 2x$

$$L(w) = \frac{1}{3}[(w - 2)^2 + (2w - 4)^2 + (3w - 6)^2]$$

一條最低點在 $w = 2$ 的 **拋物線**

從 $w^{(0)} = 0$ 走一步 ($\eta = 0.01$)

$$\begin{aligned}\frac{dL}{dw}(w^{(0)}) &= \frac{2}{3} \sum_i (w^{(0)}x_i - y_i)x_i \\ &= \frac{2}{3}[(-2) \cdot 1 + (-4) \cdot 2 + (-6) \cdot 3] = -56/3\end{aligned}$$

$$w^{(1)} = w^{(0)} - 0.01 \cdot (-56/3) \approx 0.187$$

$w^{(1)}$ 比 $w^{(0)}$ 更靠近 2，loss 下降 ✓

✓ Tip

$L : \mathbb{R}^n \rightarrow \mathbb{R}$ 可微時，**gradient** 是 L 在 θ 處所有偏導數依序排成的向量

$$\nabla L(\theta) = \left(\frac{\partial L}{\partial \theta_1}(\theta), \dots, \frac{\partial L}{\partial \theta_n}(\theta) \right) \in \mathbb{R}^n。$$

Learning rate：第一個重要的 hyperparameter

 η 太小

loss 降得 **很慢**

訓練半天沒動靜

 η 剛好

穩定下降

通常要試出來

 η 太大

震盪或發散

一步跨過谷底，越彈越高

Info

對數列 $\theta^{(t)}$ 來說：

- η 太小 $\Rightarrow$ 每步 $\theta^{(t+1)} - \theta^{(t)}$ 都很小
- η 太大 $\Rightarrow$ 可能跨過最低點，出現 $L(\theta^{(t+1)}) > L(\theta^{(t)})$ ，數列不再單調遞減，甚至發散。

Chain Rule

單變數 chain rule 的推導

設 f, g 可微， $y = f(u)$ 、 $u = g(x)$ ： x 變化 Δx 時，

$$\Delta u \approx \frac{du}{dx} \Delta x, \quad \Delta y \approx \frac{dy}{du} \Delta u$$

把第一式代入第二式， u 的變化量直接換成 x 的變化量：

$$\Delta y \approx \frac{dy}{du} \cdot \frac{du}{dx} \Delta x$$

兩邊除以 Δx ，取 $\Delta x \rightarrow 0$ 的極限，左邊剛好是 y 對 x 的導數定義，於是得到合成函數 $y = f(g(x))$ 也可微，且

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

本質：兩段線性近似「串接」起來，放大率直接相乘。

Chain Rule

$L(\theta^{(t)})$ 是多變數合成

現在 L 不是只吃一個中間變數，是吃 k 個： $L = f(u_1, \dots, u_k)$ ，每個 $u_i = g_i(\theta)$ 皆可微。多變數可微：

$\exists \nabla L(u) \in \mathbb{R}^k$ ，使得對任意 $h \in \mathbb{R}^k$ ，

$$L(u + h) = L(u) + \nabla L(u) \cdot h + o(\|h\|), \quad \nabla L(u) \cdot h = \sum_{i=1}^k \frac{\partial L}{\partial u_i} h_i$$

令 $h_i = \Delta u_i$ ，則 $\Delta L = \sum_{i=1}^k \frac{\partial L}{\partial u_i} \Delta u_i + o(\|\Delta u\|)$ ；當 $\Delta u \rightarrow 0$ ， $o(\|\Delta u\|)$ 項消失得比線性項快，一階近似只剩

$$\Delta L \approx \sum_{i=1}^k \frac{\partial L}{\partial u_i} \Delta u_i$$

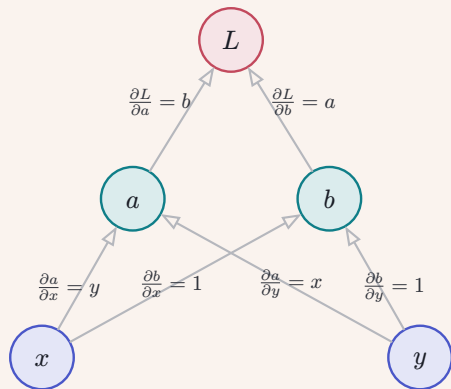
再把每條路徑自己的線性近似 $\Delta u_i \approx \frac{du_i}{d\theta} \Delta \theta$ 代進去、除以 $\Delta \theta$ 、取極限：

$$\frac{dL}{d\theta} = \sum_{i=1}^k \frac{\partial L}{\partial u_i} \cdot \frac{du_i}{d\theta}$$

這就是 $\nabla L(\theta^{(t)})$ 每個分量 $\frac{\partial L}{\partial \theta_i}$ 的算法：每條路徑先各自「一路相乘」， k 條路徑再「全部相加」。

Autograd ①：forward 留下一張計算圖

計算圖： $a = xy$ ， $b = x + y$ ， $L = ab$



代入 $x = 2$ 、 $y = 3$ ： $a = 6$ ， $b = 5$ ， $L = 30$

forward pass 順便記下兩件事

每執行一個運算就留下節點與邊：**節點** 記算出的值，**邊** ($u \rightarrow v$) 記局部偏導在 **當下數值** 的值：

邊	局部偏導	記下的數值
$x \rightarrow a$	$\frac{\partial a}{\partial x} = y$	3
$y \rightarrow a$	$\frac{\partial a}{\partial y} = x$	2
$x \rightarrow b$	$\frac{\partial b}{\partial x} = 1$	1
$y \rightarrow b$	$\frac{\partial b}{\partial y} = 1$	1
$a \rightarrow L$	$\frac{\partial L}{\partial a} = b$	5
$b \rightarrow L$	$\frac{\partial L}{\partial b} = a$	6

x 、 y 各分 **兩條路徑** 流向 L ，上一頁「多路徑」的最小例子。

Autograd ② : reverse-mode 演算法本體

Backward(G, L) : DAG 上的一次反向掃描

```
Backward(G, L):  
  for each node v in G:  
    grad[v] ← 0  
  grad[L] ← 1  
  for v in ReverseTopoOrder(G):  
    for each edge (u → v) in G:  
      grad[u] ← grad[u] + grad[v] · (∂v/∂u)  
  return grad
```

每一行都來自上一頁的 chain rule

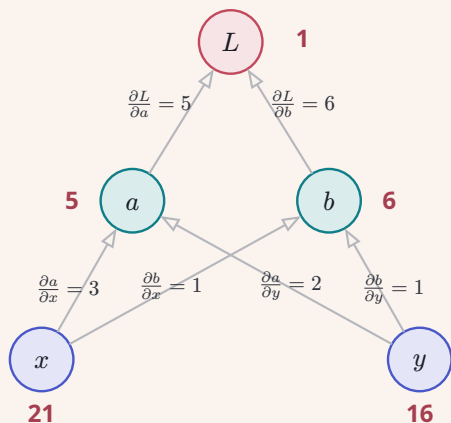
- $\text{grad}[L] \leftarrow 1$: 遞推起點, $\frac{\partial L}{\partial L} = 1$
- `ReverseTopoOrder` : v 的 **所有下游** 都處理完, 才輪到 v 被造訪
- $\text{grad}[v] \cdot (\partial v / \partial u)$: 單一路徑 **一路相乘**
- $\text{grad}[u] + \dots$: 多條路徑 **全部相加**

✓ Tip

迴圈不變量 : v 被造訪的那一刻, $\text{grad}[v]$ 已經等於 $\frac{\partial L}{\partial v}$, 所有經過 v 的路徑貢獻都累加完畢, 拿它往上游推才是安全的。

Autograd ③：把演算法在圖上跑一遍

同一張圖，紅字＝跑完的 grad



逐步執行：三次造訪、六條邊

造訪	沿邊	更新 $\text{grad}[u] \leftarrow \text{grad}[u] + \text{grad}[v] \cdot \frac{\partial v}{\partial u}$	結果
L	-	初始化 $\text{grad}[L] \leftarrow 1$	1
L	$a \rightarrow L$	$\text{grad}[a] \leftarrow 0 + 1 \times 5$	5
L	$b \rightarrow L$	$\text{grad}[b] \leftarrow 0 + 1 \times 6$	6
a	$x \rightarrow a$	$\text{grad}[x] \leftarrow 0 + 5 \times 3$	15
a	$y \rightarrow a$	$\text{grad}[y] \leftarrow 0 + 5 \times 2$	10
b	$x \rightarrow b$	$\text{grad}[x] \leftarrow 15 + 6 \times 1$	21
b	$y \rightarrow b$	$\text{grad}[y] \leftarrow 10 + 6 \times 1$	16

x 、 y 各被寫入 **兩次**，「多路徑相加」在演算法裡自動發生。

驗算： $L = x^2y + xy^2$ ，直接微分得 $\frac{\partial L}{\partial x} = 2xy + y^2 = 21 \checkmark$ 、 $\frac{\partial L}{\partial y} = x^2 + 2xy = 16 \checkmark$ 。

Autograd ④ : Engine 真正怎麼跑

真實做法：入度計數 + ready queue

```

compute_dependencies(G, L):
    for each node v in G:
        deps[v] ← 0
    for each edge (u → v) in G:
        deps[u] ← deps[u] + 1

Engine(G, L):
    queue ← [L]
    while queue not empty:
        v ← queue.pop()           # 順序任意
        run v.backward()          # 把 grad[v] 推給每個 u
        for each edge (u → v) in G:
            deps[u] ← deps[u] - 1
            if deps[u] == 0:
                queue.push(u)
  
```

跟 ② 是同一個演算法

- `deps[u]` = u 的 **out-degree**：有幾個下游就要等幾次貢獻到齊
- `deps[u] = 0` 才進 queue，等價於 ② 的 **ReverseTopoOrder**：用 Kahn's algorithm 動態算出來，不是先排好一份 list
- queue 裡不只一個 ready 節點時，彼此順序任意，甚至可以 **平行跑**
- 圖是每次呼叫 `backward()` 才現場建的，`deps` 也是現場算，動態圖由此而來

Autograd ⑤：為什麼是 reverse、不是 forward？

➡ forward-mode

固定 **一個輸入** θ_i ，順著圖正向掃一遍，得到所有節點對 θ_i 的導數。

湊齊 ∇L 的 n 個分量：掃 **n 遍**

← reverse-mode

固定 **一個輸出** L （純量），反向掃一遍，同時得到 L 對 **所有輸入** 的偏導。

整個 ∇L ：掃 **1 遍**，成本 $\approx$ 一次 forward

① Info

訓練第 t 步呼叫 `loss.backward()`，跑的就是這個演算法；算出的 $\nabla L(\theta^{(t)})$ 交給 `optimizer.step()` 產生 $\theta^{(t+1)}$ 。

I 訓練 loop

```
train.py

1 optimizer.zero_grad()      # 清掉上一輪梯度
2 pred = net(xb)             # forward：算預測
3 loss = loss_fn(pred, yb)    # loss：算錯誤
4 loss.backward()            # backward：算每個參數的梯度
5 optimizer.step()           # step：更新參數
```



Mini-batch SGD

完整 loss vs mini-batch 估計

$$L(\theta) = \frac{1}{N} \sum_i \ell_i(\theta)$$

$$\nabla L(\theta) \approx \frac{1}{|B|} \sum_{i \in B} \nabla \ell_i(\theta)$$

資料一大，全算太慢 → 每次 **抽一小批**

統計性質

- 均勻抽樣時：完整梯度的 **無偏估計**
- 但有 **變異數** → 訓練曲線會抖
- batch 越小，變異數越大，曲線越毛

✓ Tip

抖動不是 bug，它是 mini-batch 訓練的自然結果。看到毛毛的 loss curve 不要慌。

I 泛化：training set 之外的世界

把 *training loss* 壓低不是終點。機器學習真正關心的是 **泛化**：沒看過的新資料會不會做。

 **training set**

用來 **更新參數**

 **validation set**

不更新參數，只檢查新資料表現

① Info

實務規則：用 training set 訓練；用 validation curve 決定 **何時停、選哪個 checkpoint、調哪組 hyperparameter**。

Overfitting

訓練時要盯的兩條曲線

training loss	持續下降 ↘
---------------	--------

validation loss	先下降， 後來上升 ↘ ↗
-----------------	----------------------

岔開的位置，就是該停的位置

overfitting 的診斷

大 MLP：training accuracy 100%，
validation accuracy 停住甚至下降

模型不是學到穩定規律，
是在 **背訓練資料**

⚠ 注意

BC 的 rollout drift 可以看成 **泛化失敗的時間版本**，Part 2 會正面遇到它。

模型是函數，loss 是地形，
gradient 是下坡方向，
optimizer 是走路規則，
validation 是檢查是不是只背了訓練資料。

SECTION 04

Part 2 · 序列化 BC



04

Lab 2 : Push-T 的 action 序列



`basic_sequence_bc · lerobot/
pusht_keypoints`

Push-T：人控制推桿，把 **T 形物塊** 推到目標位置。

這一段先拿掉 image，**只留下 action 序列**。

autoregressive

一段示範： $a_1, a_2, \dots, a_T$

要學：給定 $a_1 \dots a_t$ ，預測 a_{t+1}



自回歸分解：一條恆等式

$$p(a_1, \dots, a_T) = \prod_{t=1}^T p(a_t \mid a_{1:t-1})$$

恆等式，不是假設

由條件機率定義 $p(x, y) = p(x) p(y \mid x)$

對 $a_1, \dots, a_T$ 反覆展開即得。

對 **任何** 聯合分布都成立，沒有丟失資訊。

模型化發生在「條件集」

bigram	$\pi_{\theta}(a_{t+1} \mid a_t) \cdot \text{截斷到 1 步}$
Transformer	$\pi_{\theta}(a_{t+1} \mid a_{1:t}) \cdot \text{整段前綴}$

近似的好壞 = 截斷丟掉的條件相關性多寡

① Info

NLL 跟著分解： $-\log p(a_{1:T}) = \sum_{t=1}^T -\log p(a_t \mid a_{1:t-1})$ ，整段序列的 loss 是 **每步條件 NLL 的和**，這條等式是之後 teacher forcing 的數學根據。

| Action tokenization : 連續 → 離散

原始 action

連續座標 (x, y) ，範圍約 $0 \sim 512$

離散化：每軸切 16 格

$\text{vocab_size} = 16 \times 16 = 256$ ，每個 $(\text{bin}_x, \text{bin}_y)$
對應一個 token id

① Info

action 落在哪格就變成哪個 token，Transformer 最常見的訓練形式是分類：**預測下一個 token 是 vocab 裡哪一個**。

量化誤差可以算

格寬 $\Delta = 512/16 = 32$ ；還原到格子中心，每軸誤差至多 $\Delta/2 = 16$ ；bin 內均勻分布時量化 $\text{MSE} = \Delta^2/12$

每軸切 k 格的兩頭代價

誤差 $\propto 1/k$ 下降；但 $\text{vocab} = k^2$ 成長、每個 token 平均樣本數 $\propto 1/k^2$ 掉，**分類問題變難**

Bigram：最小的序列模型

Markov 假設：條件集截斷到 1

$$\pi_{\theta}(a_{t+1} \mid a_{1:t}) := \pi_{\theta}(a_{t+1} \mid a_t)$$

參數就是一張 256×256 的表：

```
table[current_token] → logits over next
```

它的 MLE 有封閉解

$$\hat{\pi}(j \mid i) = \frac{N(i,j)}{\sum_{j'} N(i,j')}$$

$N(i, j)$ = 資料裡 $(i \rightarrow j)$ 出現次數。
數次數、逐列正規化，就是 NLL 的全域最小值。

天花板寫在假設裡：被截斷丟掉的 $a_{1:t-1}$ ，任何訓練都救不回來。

Embedding：查表就是矩陣乘法

one-hot × E = 取出第幾列

token 3，vocab_size = 6：

$[0, 0, 0, 1, 0, 0] \cdot E = E$ 的第 3 列

embedding table 是一個 **可訓練矩陣**，
查表本質上還是矩陣乘法

神經版 bigram = 低秩分解

$\text{logits}(i) = e_i^\top EW$, $E \in \mathbb{R}^{256 \times C}$, $W \in \mathbb{R}^{C \times 256}$

整張 256×256 logit 表被分解成 **秩 $\leq C$** 的乘積
參數 $2 \cdot 256C$ vs 256^2 ： $C = 32$ 時只剩 1/4

① Info

$C \geq 256$ 可精確重現計數表； $C < 256$ 是壓縮，低秩約束下要壓低 NLL，轉移行為相近的 token **必須共用方向**。「相似 token 有相似 embedding」是最佳化的結果，不是願望。

Shape discipline : B、T、C

序列模型的三個字母

B	batch size
T	sequence length
C	channel / embedding dim

常見形狀

tokens	(B, T)
embedding	(B, T, C)
logits	(B, T, vocab_size)

✓ Tip

遇到 shape mismatch，**不要猜**，先 `print(x.shape)`。深度學習除錯很大一部分是把形狀帳對齊。

Attention：可學習的加權平均

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

$$Q, K \in \mathbb{R}^{T \times d_k}, \quad V \in \mathbb{R}^{T \times d_v}, \quad QK^\top \in \mathbb{R}^{T \times T}, \quad \text{輸出} \in \mathbb{R}^{T \times d_v}$$

$QK^\top$

query 跟每個 key 的 **內積**
內積 = 相似度，第三次出現

$\div \sqrt{d_k}$

分量獨立、零均值、單位變異數時，內積變異數是 d_k
不縮放 $\rightarrow$ softmax 飽和 $\rightarrow$ 梯度消失

$\text{softmax} \cdot V$

每列變成加總為 1 的權重
拿權重對 value **加權平均**

📌 Info

核心想法：不硬選某一個過去位置，而是對 **所有可見位置做加權平均**，權重由模型自己算。

三個 token 的手算

scores = QK^T

$$\begin{pmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}$$

softmax 後的 weights

$$\begin{pmatrix} 0.67 & 0.24 & 0.09 \\ 0.09 & 0.67 & 0.24 \\ 0.21 & 0.21 & 0.58 \end{pmatrix}$$

第一列： $(e^2, e^1, e^0) =$

$(7.39, 2.72, 1.00)$ ，

除以和 11.11 $\rightarrow (0.67, 0.24, 0.09)$

第一個輸出

$$0.67v_1 + 0.24v_2 + 0.09v_3$$

value 的加權平均

attention 是 可學習的加權平均。

Transformer block 的其他零件

position embedding

$\text{Attn}(PX) = P \text{Attn}(X)$ ：重排輸入，輸出跟著重排
→ 改餵 $x_t + p_t$ ，順序資訊才進得了模型

multi-head

$\text{head}_h = \text{Attn}(XW_Q^h, XW_K^h, XW_V^h)$
各 head 在 $d_k = C/H$ 維子空間，concat 乘 W_O

residual + LayerNorm

$y = x + f(x) \Rightarrow \partial y / \partial x = I + \partial f / \partial x$ ，連乘每項含 I
LN： $\gamma(x - \mu) / \sigma + \beta$ ，逐 token 定住激活量級

MLP block

$W_2 \sigma(W_1 x)$ ，逐 token 套用
attention 跨位置交換資訊，MLP 在 channel 維加工

⚠ 注意

序列 action **一定要有 position embedding**，沒有它， a_3 和 a_7 對調，輸出只是跟著對調。

Causal mask：不准偷看未來

加法型 mask

$$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)$$

$$M_{ts} = \begin{cases} 0 & s \leq t \\ -\infty & s > t \end{cases}, \quad M = \begin{pmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{pmatrix}$$

$e^{-\infty} = 0 \Rightarrow$ 未來位置權重 **恰為 0**，
每列仍在 $s \leq t$ 上正規化、加總為 1



為什麼是底線

訓練時一次把整段 sequence 丟進模型，
但位置 t 預測 a_{t+1} 時 **不能看之後的答案**。

偷看未來 $\rightarrow$ 訓練 loss 很好看

$\rightarrow$ 部署立刻露餡（rollout 時 **未來還沒發生**）

Teacher forcing：平行訓練所有位置

● ● ● training pairs

```
input : a_1, a_2, ..., a_t  
target: a_2, a_3, ..., a_{t+1}
```

$$-\log \pi_{\theta}(a_{2:T+1} \mid a_1) = \sum_{t=1}^T -\log \pi_{\theta}(a_{t+1} \mid a_{1:t})$$

訓練時

每一項都條件在 **真實前綴** 上；causal mask 讓一次 forward **同時算出全部 T 項**

rollout 時

模型吃到的是 **自己前一步生成的 action** → 前一步錯了，輸入就不再像訓練資料

⚠ 注意

錯誤可能一路累積，這叫 **distribution shift** 或 **compounding error**。

ALVINN：1989 年就撞過的牆

CMU · 1980 年代末

用神經網路開真車。

訓練資料：人類 **正常開在車道中央** 的
影像 + 方向盤操作。

問題

人類示範裡很少有

「車已經偏出車道，**該怎麼救回來**」的資料。

一旦偏掉 → 進入沒覆蓋的 state → 更容易再
錯

錯一步



進入陌生 state



更容易再錯



state 更陌生

錯誤的複利： $O(T\varepsilon)$ vs $O(T^2\varepsilon)$

一般 supervised

錯誤 **不影響**後續輸入

$$O(T\varepsilon)$$

T 步期望累積 cost 線性成長

BC rollout · Ross & Bagnell 2010

每個錯誤把 state **帶離訓練分布**

$$O(T^2\varepsilon)$$

骨架：第 t 步首次犯錯機率 $\leq \varepsilon$ ，之後最壞每步 cost 1、共 $T - t + 1$ 步

$$\mathbb{E}[\text{cost}] \leq \sum_{t=1}^T \varepsilon(T - t + 1) = \varepsilon \frac{T(T+1)}{2}$$

✓ Tip

Dagger (Ross, Gordon & Bagnell 2011)：讓模型 rollout、收集它闖進的怪 state、問老師「這裡該怎麼做」、加回訓練集，可壓回 $O(T\varepsilon)$ 量級。

Temperature：推論時的旋鈕

$$p_i(\tau) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

兩兩比值 $p_i/p_j = \exp((z_i - z_j)/\tau)$ ，全部行為由它決定

$\tau \rightarrow 0^+$

收斂到 **argmax**

$z_i < z_{\max}$ 時 $(z_i - z_{\max})/\tau \rightarrow -\infty$ ，比值 $\rightarrow 0$

$\tau = 1$

原始 softmax

照模型信心抽樣

$\tau \rightarrow \infty$

趨近 **uniform**

所有比值 $\exp((z_i - z_j)/\tau) \rightarrow e^0 = 1$

× 危險

機器人 policy 通常不能太隨機，每個抖動都可能變成 **真實世界的碰撞**。

SECTION 05

Part 3 · 視覺表示



05

Lab 3：輸入是一張圖

 **basic_vision_representation · lerobot/
pusht_image**

任務仍是 Push-T，這次：

96×96 RGB image_t → action_t (x, y)

先不處理 action history、不處理語言

專心問一件事

影像怎麼變成
policy 能用的 **representation** ？

96×96×3

輸入像素

27648

攤平後的數字量

2

輸出維度 (x, y)

為什麼不能直接攤平

圖像有強規律

- 相鄰像素通常 **相關**
- 同一個局部形狀可能出現在 **不同位置**
- 小邊緣 → 角 → 部件 → 物體
- 物體位置會影響 action

兩條路把規律用起來

CNN：把規律 **寫進架構**

(局部性、weight sharing)

ViT：把圖切成 patch tokens，
用 attention **學** patch 之間的關係

⚠ 注意

攤平丟進 MLP 不是不能算，是 **浪費了圖像結構**

一個 3×3 kernel 的手算

灰階圖：左黑右白

```
0 0 0 1 1 1
0 0 0 1 1 1
0 0 0 1 1 1
0 0 0 1 1 1
```

垂直邊緣 kernel

```
-1 0 1
-1 0 1
-1 0 1
```

滑過去看內積

- 全黑區：內積 = 0
- **黑白交界：內積變大 ✨**
- 全白區：正負抵消 → 0

kernel 是 **局部模板比對器**：滑過整張圖，看到像自己的局部形狀就亮。

卷積公式、cross-correlation 與 weight sharing

完整公式

$$y[u, v, c_{\text{out}}] = \sum_{i, j, c_{\text{in}}} K[i, j, c_{\text{in}}, c_{\text{out}}] x[u + i, v + j, c_{\text{in}}]$$

名字的小秘密

嚴格說這是 **cross-correlation**：訊號處理的 convolution 會先翻轉 kernel。深度學習一律用不翻轉版，kernel 是學出來的，翻不翻只差重新參數化。

weight sharing $\Rightarrow$ 平移等變

令平移 $(T_\delta x)[u] = x[u - \delta]$ ，代入公式即得

$$(T_\delta x) \star K = T_\delta(x \star K)$$

輸入平移，feature map **跟著平移**，「左上角有用、右下角也有用」的精確版。

① Info

blur / sharpen / edge kernel 是 **人設計的**；CNN 的關鍵：kernel 數字 **由梯度下降學出來**。

SmallCNN：解析度換 channel

四層 stride=2



receptive field 可遞推： $r_l = r_{l-1} + (k - 1)j_{l-1}$ ，

$$j_l = sj_{l-1}$$

3×3、stride 2 四層： $r : 1 \rightarrow 3 \rightarrow 7 \rightarrow 15 \rightarrow 31$ ，末層一格看 **31×31** 像素

最後 `AdaptiveAvgPool2d(1)`： $6 \times 6 \times C \rightarrow C$ 維向量 → 線性層 → action

lab 的三個詞

stride	每次滑幾格； <code>stride=2</code> 寬高約減半
padding	邊界補 0，控制輸出大小、讓邊緣被掃到
channel	RGB 3 個 in；卷積層可輸出多個 feature channels

Encoder 的觀念

$$\pi_{\theta} = f_{\psi} \circ \varphi_w, \quad \varphi_w : \mathbb{R}^{27648} \rightarrow \mathbb{R}^C, \quad f_{\psi} : \mathbb{R}^C \rightarrow \mathbb{R}^2$$



端到端 = 一條 chain rule

$$\nabla_w L = \frac{\partial L}{\partial f} \frac{\partial f}{\partial z} \frac{\partial z}{\partial w}$$

loss 的梯度 **穿過 f_{ψ} 傳回 encoder**，
representation 是被 action loss 逼出來的，不是人工特徵

VLA 裡的 vision encoder

同一個角色，只是更大、資料更多、
輸出從一個向量變成一串 tokens
($\mathbb{R}^C \rightarrow \mathbb{R}^{M \times C}$)

| Saliency：看模型對哪些像素敏感

input-gradient saliency

$$\text{saliency}[\text{pixel}] = |\partial \text{scalar} / \partial \text{pixel}|$$

根據是一階 Taylor： $g(x + \delta) = g(x) + \nabla_x g \cdot \delta + o(\|\delta\|)$ ，saliency 讀的是 **線性項的係數大小**。

輸出是 2 維 action，梯度要對 **scalar** 取 →
lab 取 `f(x).abs().sum()`（預測 action 的 L1 norm）；

對每像素求偏導、取絕對值、RGB 三 channel 平均 →
一張 H×W 熱圖

是 debug 工具，不是真理

✓ 提醒模型可能在看推桿、T 塊，
或 **背景捷徑**

✗ 只回答局部問題：「目前這個輸入附近，
哪些像素影響大？」

ViT：把圖像變成 patch tokens

patchify

96×96 圖、`patch_size=12`：

$$8 \times 8 = 64 \text{ patches}$$

每個 patch 攤平成 $12 \cdot 12 \cdot 3 = 432$ 維，
乘 $W_E \in \mathbb{R}^{432 \times C} \rightarrow \text{embedding}$
，還是 Part 1 那行 W_x

token 序列



`CLS` token 負責 **總結整張圖**；
加上 position embedding 後進 self-attention
，公式跟 Part 2 一字不差， 65^2 對相似度

Info

到這裡，「圖像」跟「序列」在模型眼裡已經是同一種東西：**一串 tokens**。

Mask 取決於資料，不取決於 Transformer

時間序列 · causal mask

時間有「未來」，部署時未來還沒發生

→ 要遮

$$\begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$

圖像 patch · full self-attention

patch 7 看 patch 42 不叫偷看答案

→ 不遮，每個 patch attend to 每個 patch

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Transformer $\neq$ *language model*，attention 不一定要 *causal mask*。
mask 取決於 資料關係。

Pretraining：真實 VLA 不從零學視覺

兩階段最佳化，同一組參數

- ① $w^* = \arg \min_w L_{\text{pre}}(w; D_{\text{pre}})$
- ② 從 (w^*, ψ_0) 出發 minimize $L_{\text{BC}}(w, \psi; D_{\text{robot}})$

fine-tune 改的是 **初始點**，不是 loss 的定義，
第二階段仍是 Part 0 那條 BC loss

為什麼非這樣不可

$ w $	encoder 參數 $\sim 10^8 \sim 10^9$
$ D_{\text{pre}} $	圖文對 $\sim 10^9$
$ D_{\text{robot}} $	軌跡 $\sim 10^3 \sim 10^6$

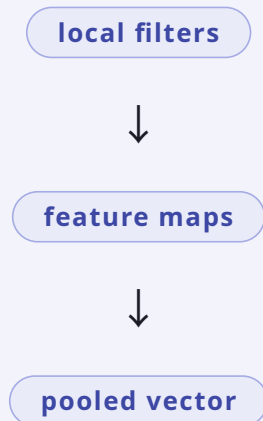
每組參數都要樣本約束（Part 3 開頭那筆帳）
→ 億級參數的 w **只有 D_{pre} 付得起**

① Info

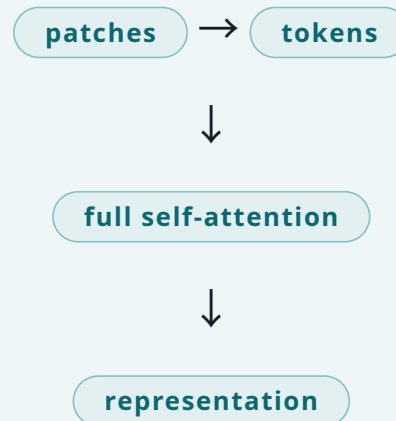
lab 從亂數練小 CNN/ViT 是教學設定。真實 VLA：pretraining 決定 w 的起點，robot 資料只負責把表示 **接到 action**。

Part 3 收束

CNN



ViT



兩者都是把 **raw pixels** 轉成 policy 可用的 **representation**。

SECTION 06

Part 4 · VLA-shaped BC



06

| Homework : smol-libero



HuggingFaceVLA/smol-libero

影像 + 語言指令 + **7 維機械手臂 action**

$$\pi_{\theta}(a_{t+1} \mid \text{image}_t, \text{instruction}, a_{1:t})$$

image + instruction + action_history →
next_action

定位要放對

✗ 不是要訓練可部署機器人

✓ 是讓你 **看懂 VLA 的資料流**

Info

它仍然是 BC。只是 context 變大了。

三條輸入：vision、language、action history

Vision

image $\rightarrow v_1, \dots, v_M$

經過 vision encoder
變成 patch / feature tokens

回答：**我看到了什麼？**

Language

“put the red block on the plate”

$\rightarrow l_1, \dots, l_L$

文字進模型前也要
tokenization

回答：**人叫我做什麼？**

Action history

$a_1, \dots, a_t$

tokenized 或投影成
action embeddings，丟進
decoder

回答：**我做到哪一步了？**

✓ Tip

進 decoder 前全部變成同一種東西：條件集是一串 tokens $[v_{1:M}; l_{1:L}; a_{1:t}]$ ，長度 $M + L + t$ ，拿掉哪一段， π_θ 的條件就少哪一種依據。

文字 token 與 action token 完全同型

文字



$e_i^\top E_{\text{text}}, E_{\text{text}} \in \mathbb{R}^{V_{\text{text}} \times C}$ · vocab 來自 **tokenizer**

action

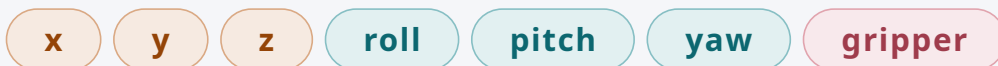


$e_j^\top E_{\text{act}}, E_{\text{act}} \in \mathbb{R}^{256 \times C}$ · vocab 來自 **離散化**

🗺️ 把 LLM 放回地圖

ChatGPT 的 base model = Part 2 的「用過去 token 預測下一個 token」：token 換成文字、資料換成大規模文本、模型放大很多。產品化前還有 instruction tuning、RLHF 等 post-training，但那改變的是它 **偏好輸出什麼**，autoregressive 骨架不變。

7 維 action 怎麼 tokenized



硬合成一個聯合 token

$$\text{vocab} = \text{bins}^7 = 32^7 \approx 3.4 \times 10^{10}$$

大到 不可用

homework : 每軸各自離散化

軸 $d \rightarrow \text{token} \in \{0, \dots, 31\}$

輸出形狀 $(B, T, 7, \text{bins_per_dim})$

loss = 7 個軸的 cross-entropy 平均

$$32^7 \approx 3.4 \times 10^{10}$$

聯合 vocab

$$7 \times 32 = 224$$

per-axis 輸出

Per-axis 分解：假設了什麼、換到了什麼

$$p(a \mid \text{context}) = \prod_d p(a_d \mid \text{context})$$

假設：給定 context 後各軸條件獨立

不完全成立，接近物體時
z 和 gripper 顯然相關

為什麼還是划算

相關性有一部分已被 **context 吸收**；
剩下的建模損失，換來輸出空間
從 32^7 降到 7×32

✓ **Tip**

這是一次 tradeoff 分析，讀論文時這部份很重要。

Action chunking：看一眼，做一段

training sample

```
input actions : a_t,      a_{t+1}, ..., a_{t+B-1}
target actions: a_{t+1}, a_{t+2}, ..., a_{t+B}
```

chunk 內第 k 步： $\pi_{\theta}(a_{t+k+1} \mid \text{image}_t, \text{instr}, a_{1:t+k})$ ，影像下標 **凍結在 t**

買到的：計算省 B 倍

每步重看：一個 chunk 跑 **B 次** vision encoder

開頭看一次：只跑 **1 次**

encoder 是全模型最貴的零件，這筆帳直接除以 B

付出的：畫面過時 k 步

真實 state 已走到 $t + k$ ，條件裡的影像還停在 t

→ 這 k 步的動態 **只能靠 $a_{t:t+k}$ 回推**，
 B 越大、要回推的越多

Cross-attention : decoder 查詢多模態 context

Q 來自 action , K/V 來自 context

$$\text{context} = [v_1, \dots, v_M, l_1, \dots, l_L]$$

$$Q = \text{action } W_Q, \quad K = \text{context } W_K, \quad V = \text{context } W_V$$

$$\text{CrossAttn} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

$$Q \in \mathbb{R}^{T_a \times d_k}, \quad K, V \in \mathbb{R}^{(M+L) \times d_k} \rightarrow \text{權重} \in \mathbb{R}^{T_a \times (M+L)}, \text{每列在 context 上加總為 1}$$



心智圖

action decoder 在 **問問題**，
影像和語言提供 **資料庫**。

cross-attention 權重 = 模型當下
從 context **借資訊的分配**

Info

某一步 action 問：「我現在該往哪裡移動？」影像 patch 給物體位置、語言 token 給任務目標、歷史 action 給目前進度。

一層 VLA decoder 的三件事

① causal self-attention

action step t attends to
steps $\leq t$

跟 Part 2 一樣：
保持時間方向，**不偷看未來**

② cross-attention

action tokens 查詢
vision / language context

借外部資訊

③ MLP

對每個 token 自己的表示
做非線性加工

attention 交換資訊，
MLP 加工資訊

✓ Tip

兩種 mask 同一層並存，寫成同一個 M ：action 列對 context 的 $M + L$ 欄全為 0（可見）；對 action 欄沿用 Part 2 的 $M_{ts} = -\infty (s > t)$ ，Part 3 那條原則的實戰版。

I 名字叫 VLA，訓練目標還是 BC

minimize $-\log \pi_{\theta}(\text{demonstrator_action} \mid \text{image}, \text{instruction}, \text{action_history})$

變了的

- 名字：VLA
- 程式碼：vision encoder、language encoder、action decoder
- loss：(B*T*7) 個分類問題一起算

沒變的

訓練目標仍是：
示範者做了什麼，
就把那個答案的機率拉高。

這就是 BC。

Report 裡的三張圖各自證明什麼

面板	告訴我們什麼	對應的理解
Loss / Action L2	訓練 有沒有動	Part 1 的 loss 與曲線
prediction panel	模型 輸出什麼	Part 2 的 token 分布
cross-attention panel	decoder 比較 讀影像還是語言	Part 4 的 cross-attention

❶ Info

這些圖不是為了證明 homework 訓練出可部署 policy，而是證明 **你看得懂資料流和訓練迴圈**。

SECTION 07

課後延伸



07

補充教材



3Blue1Brown

神經網路系列



Karpathy

“Neural Networks: Zero to Hero”



李宏毅

機器學習（台大公開課）



CS285

Berkeley Deep RL (Sergey Levine)



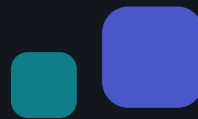
ACT

Zhao et al. 2023，論文原文



OpenVLA

Kim et al. 2024，論文原文



Q & A

獵奇緯 · Summer Course